ASHESI UNIVERSITY

PharmaVault

CS 443: MOBILE DEVELOPMENT

Roseline Dzidzeme Tsatsu

Lecturer: Mr. David A. Sampah

14th April 2026

PharmaVault - Mobile Application Development Report

Activity 1: Planning, Case Scenario and Contract

1.1. Background Information

Access to medicines in Ghana and across sub-Saharan Africa is fragmented. Patients often visit multiple pharmacies looking for a specific drug, compare prices verbally, or rely on word-of-mouth to find nearby pharmacies with stock. There is no centralised, real-time digital channel connecting patients to verified pharmacies, their inventories, and their pricing.

PharmaVault was conceived as a direct solution to this problem. It is a mobile application built for patients and healthcare consumers in Ghana that allows them to browse medicines, compare prices across partner pharmacies, upload prescriptions for fulfillment, add medicines to a cart, and place delivery orders, all from a single app. Partner pharmacies are onboarded as profiles on the platform and manage their own product listings and pricing.

The target organisation is any pharmacy chain, independent pharmacy, or patient-facing health service operating in Ghana. The application is designed for individual patient use. Its principal purpose is to reduce the time and cost of accessing medicines by making pharmacy inventory and pricing transparent and delivery-enabled.

1.2. Target Market

PharmaVault is designed to serve the following user segments:

- Individual patients and healthcare consumers in Ghana
- Patients managing chronic conditions such as diabetes, hypertension, and malaria who need regular medicine refills
- Caregivers purchasing medicines on behalf of family members
- Patients in urban and peri-urban areas with limited mobility
- Partner pharmacies seeking a digital storefront and order management channel

1.3. Principal Services the App Provides

The table below summarises the principal services that PharmaVault provides to its users.

Service
Browse medicines by category, brand, or keyword search
View real-time pharmacy pricing and stock levels per product
Compare prices across multiple pharmacies for the same medicine
View partner pharmacy profiles with location, contact, and operating hours
GPS-based distance display from user to each pharmacy
Upload prescriptions for pharmacist review and fulfillment
Add medicines to a cart and place delivery orders
Track order status in real-time: Pending, Confirmed, Dispatched, Delivered

Service
Receive in-app push notifications on order and prescription status changes
Manage profile, delivery address, and contact information
Secure authentication via email and password or Google Sign-In

1.4. Functional and Non-Functional Requirements

1.4.1. Functional Requirements

The functional requirements define the specific behaviours the application must support. The table below lists all functional requirements for PharmaVault.

Functional Requirement
The app shall allow a user to create an account and log in securely
The app shall support Google Sign-In as an authentication option
The app shall display a browsable, searchable catalogue of medicines
The app shall show which pharmacies carry each medicine and at what price
The app shall allow users to filter medicines by category
The app shall display partner pharmacies with name, city, rating, and distance from user
The app shall allow users to add medicines to a cart and adjust quantities
The app shall allow users to place orders with a delivery address and contact
The app shall generate a unique invoice number for each order
The app shall allow users to view their order history and order details
The app shall allow users to upload a prescription image for pharmacy review
The app shall send in-app push notifications when order status changes
The app shall send in-app push notifications when prescription status is updated
The app shall display the user's real GPS location in the home screen header
The app shall open Google Maps for directions to a selected pharmacy
The app shall allow users to update their profile name, contact, and city
The app shall persist all data to Supabase across reinstalls and device changes

1.4.2. Non-Functional Requirements

The non-functional requirements define the quality and operational constraints the application must satisfy.

Non-Functional Requirement
All data storage and authentication must be handled by Supabase with no separate backend server
Authentication must be secure, and passwords must never be stored in plain text
User data must be isolated through Row Level Security, ensuring users can only access their own cart, orders, and prescriptions
The app must respond within 2 seconds for all local and cached operations
The app must work on Android 8.0 (API 26) and above
The UI must be usable on screen sizes from 5 to 7 inches
The app must gracefully handle no-internet conditions with appropriate error messages

1.5. Local Resources Used

PharmaVault makes direct use of on-device hardware and operating system resources to deliver its core functionality. Each resource requires an explicit Android runtime permission and is accessed through a dedicated Flutter package. The table below documents each local resource, the package used, and its purpose in the application.

Resource	Package Used	Usage in PharmaVault
GPS / Location	geolocator ^13.0.2	Getting the user's real-time coordinates for city display and pharmacy distance
Reverse Geocoding	geocoding ^3.0.0	Converting GPS coordinates to a human-readable city name
Camera / Gallery	image_picker ^1.1.2	Capturing or selecting prescription images for upload
Push Notifications	flutter_local_notifications ^18.0.1	Displaying order and prescription status update banners
Maps / Directions	url_launcher ^6.3.1	Opening Google Maps with navigation to a selected pharmacy
Network Images	cached_network_image ^3.4.1	Loading and caching product and pharmacy images from Supabase Storage

1.5.1. Required Android Permissions

The table below lists the Android permissions declared in AndroidManifest.xml and the purpose of each permission.

Permission	Purpose
INTERNET	All Supabase API calls, image loading, and real-time subscriptions
ACCESS_FINE_LOCATION	Precise GPS coordinates for distance-to-pharmacy calculation
ACCESS_COARSE_LOCATION	Fallback coarse location for city name display
POST_NOTIFICATIONS	Displaying order and prescription update notification banners on Android 13 and above
READ_MEDIA_IMAGES	Gallery access for selecting prescription photos
READ_EXTERNAL_STORAGE	Legacy gallery access for Android 12 and below
CAMERA	Capturing prescription photos directly from the app

Activity 2: Prototyping, Specification, Architecture and Design

2.1. Application Overview

PharmaVault follows a two-layer client-cloud architecture. The Flutter mobile app acts as the client. Supabase provides authentication, a PostgreSQL cloud database, real-time subscriptions, and file storage, eliminating the need for a separate backend server. All business logic runs inside Flutter providers and Supabase Row Level Security policies.

2.2. High-Level Architecture / Communication Flow

PharmaVault is built on a two-layer architecture. The Flutter mobile app acts as the client and communicates with Supabase directly using the Supabase Flutter SDK over REST and Realtime protocols. The diagram below illustrates how these two layers interact during a typical patient session.

[IMAGE PLACEHOLDER]
Fig 1: PharmaVault System Architecture Diagram

Fig 1. System Architecture of PharmaVault

2.3. Three-Layer Architecture

The internal architecture of PharmaVault is organised into three distinct layers. The table below describes each layer, its responsibilities, and the key technologies used.

Layer	Responsibility	Key Technologies
Presentation Layer	All UI screens, widgets, navigation flows, and user input handling	Flutter Widgets, Material 3, NavigationBar, CustomScrollView
Logic Layer	State management, Supabase queries, GPS fetching, notification triggering, cart operations, and order creation	Provider pattern with AuthProvider, ProductProvider, CartProvider, OrderProvider, LocationProvider, NotificationProvider, and PrescriptionProvider
Data Layer	Data models, Supabase reads and writes, Realtime subscriptions, SharedPreferences, and local file handling	supabase_flutter, Dart model classes, shared_preferences

2.4. Key Modules

The application is divided into functional modules. The table below describes each module and its role within PharmaVault.

Module	Description
Authentication	Email and password signup and login, Google Sign-In via google_sign_in and Supabase signInWithIdToken, and profile creation on registration
Home Screen	Greeting with real GPS city, search bar, prescription upload banner, category chips, horizontal product cards with Rx badges, and pharmacy cards with distance and delivery tags
Search Screen	Tabbed Medicines and Pharmacies search with live filtering, category filter chips, result count, and pull-to-refresh
Product Detail	Full product information, dosage, side effects, and an Available At section showing real pharmacies with per-pharmacy pricing and stock
Pharmacy Detail	Pharmacy header with rating and hours, list of pharmacy-specific products with their prices and stock counts, and a Directions button that opens Google Maps
Cart	Item listing with quantity controls, total calculation, and persistent cart state via Supabase
Checkout and Orders	Delivery address form, order placement with invoice generation, order history, and order detail with itemised breakdown
Prescriptions	Upload prescription photo from camera or gallery to Supabase Storage, and view upload history and status
Notifications	In-app notification history from Supabase Realtime, unread count badge, and mark-all-read functionality
Profile	View and update name, contact, and city, and logout
Location Service	GPS permission flow, reverse geocoding to city name, and distance calculation to pharmacies
Pharmacy-Product Linking	The pharmacy_products table links pharmacies to their products with per-pharmacy pricing and stock

2.5. Users and Use Case Diagram

PharmaVault has two actors: the Customer, who is the patient using the app, and the Pharmacy, which is a partner pharmacy whose profile and inventory is stored in the system. There is no admin interface in the mobile app. Pharmacy inventory and order status management is currently handled directly in Supabase. The diagram below maps all customer interactions to their corresponding use cases.

[IMAGE PLACEHOLDER]
Fig 2: PharmaVault Use Case Diagram

Fig 2. Use Case Diagram for PharmaVault

2.6. Application Prototype (Figma)

A high-fidelity prototype of the PharmaVault application was designed using Figma, demonstrating the interaction flow between key screens including authentication, home, search, product detail, pharmacy detail, cart, checkout, prescriptions, notifications, and profile screens. The prototype can be accessed at:

https://github.com/Rose-Alice18/PharmaVault-Mobile-App

The figure below shows selected screens from the Figma prototype.

[IMAGE PLACEHOLDER]
Fig 3: PharmaVault Figma Prototype Screens

Fig 3. The Figma Design

2.7. Database Schema (Supabase PostgreSQL)

PharmaVault uses ten tables in Supabase PostgreSQL. Row Level Security is enabled on all tables to ensure that each user can only access their own data. The tables below document every field, its data type, and its purpose.

Table: profiles

The profiles table stores information for both customer and pharmacy accounts.

Field	Type	Description
id	UUID (PK, FK to auth.users)	User identifier
customer_name	TEXT	Full name
customer_email	TEXT	Email address
customer_type	TEXT	customer or pharmacy
customer_contact	TEXT	Phone number

Field	Type	Description
customer_city	TEXT	City
customer_image	TEXT	Profile image URL
lat	DOUBLE PRECISION	GPS latitude (pharmacies only)
lng	DOUBLE PRECISION	GPS longitude (pharmacies only)

Table: categories

The categories table stores the medicine categories used to organise the product catalogue.

Field	Type	Description
cat_id	SERIAL (PK)	Category ID
cat_name	TEXT	Category name such as Pain Relief or Antibiotics

Table: brands

The brands table stores the medicine brand names associated with products in the catalogue.

Field	Type	Description
brand_id	SERIAL (PK)	Brand ID
brand_name	TEXT	Brand name such as Panadol or Ampiclox

Table: products

The products table stores the core product catalogue shared across all pharmacies.

Field	Type	Description
product_id	SERIAL (PK)	Product ID
product_title	TEXT	Medicine name and dosage
product_description	TEXT	Description
product_price	DECIMAL(10,2)	Base price in GHS
product_stock	INTEGER	Global stock count
product_image	TEXT	Image URL

Field	Type	Description
product_keywords	TEXT	Tags including rx for prescription-only products
cat_id	INTEGER (FK to categories)	Category
brand_id	INTEGER (FK to brands)	Brand

Table: pharmacy_products

The pharmacy_products table links each pharmacy to its specific product subset with per-pharmacy pricing and stock counts.

Field	Type	Description
id	SERIAL (PK)	Row ID
pharmacy_id	UUID (FK to profiles)	Pharmacy
product_id	INTEGER (FK to products)	Product
pharmacy_price	DECIMAL(10,2)	This pharmacy's price for this product
stock_count	INTEGER	This pharmacy's current stock
is_available	BOOLEAN	Whether the product is currently listed

Table: cart

The cart table stores each item a customer has added to their cart, persisted in Supabase.

Field	Type	Description
cart_id	SERIAL (PK)	Cart row ID
c_id	UUID (FK to profiles)	Customer
p_id	INTEGER (FK to products)	Product
cart_qty	INTEGER	Quantity

Table: orders

The orders table stores each order placed by a customer, including its status and delivery details.

Field	Type	Description
order_id	SERIAL (PK)	Order ID
invoice_no	TEXT	Unique invoice number formatted as INV-YEAR-MILLISECONDS
c_id	UUID (FK to profiles)	Customer
order_total	DECIMAL(10,2)	Total amount in GHS
order_status	TEXT	pending, confirmed, processing, dispatched, delivered, or cancelled
is_paid	BOOLEAN	Payment status
payment_amount	DECIMAL(10,2)	Amount paid
delivery_notes	JSONB	Address, city, and contact details
order_date	TIMESTAMPTZ	Timestamp of order placement

Table: order_items

The order_items table stores each individual product line within a placed order.

Field	Type	Description
item_id	SERIAL (PK)	Item ID
order_id	INTEGER (FK to orders)	Parent order
p_id	INTEGER (FK to products)	Product
item_qty	INTEGER	Quantity ordered
item_price	DECIMAL(10,2)	Price at time of order

Table: prescriptions

The prescriptions table tracks all prescription images uploaded by customers and their review status.

Field	Type	Description
prescription_id	SERIAL (PK)	Prescription ID
c_id	UUID (FK to profiles)	Customer
prescription_image	TEXT	Supabase Storage URL

Field	Type	Description
prescription_status	TEXT	pending, reviewing, approved, or rejected
prescription_notes	TEXT	Pharmacist notes
created_at	TIMESTAMPTZ	Upload timestamp

The figure below illustrates the Entity Relationship Diagram for the PharmaVault database.

[IMAGE PLACEHOLDER]
Fig 4: PharmaVault Entity Relationship Diagram

Fig 4. Entity Relationship Diagram

Activity 3: Implementation

3.1. Overview

PharmaVault is built entirely on Flutter and Supabase with no separate backend server. All state is managed via the Provider pattern. Supabase handles authentication, all database reads and writes, real-time order and prescription status subscriptions, and prescription image storage. The app uses the device GPS for real-time location and flutter_local_notifications for in-app notification banners.

3.2. Tools, Libraries, Frameworks, and Services

3.2.1. Mobile Frontend

The table below lists the Flutter packages used in PharmaVault, their versions, and their purposes.

Tool / Library	Version	Purpose
Flutter	3.x	Cross-platform mobile UI framework
Dart	3.x	Programming language
provider	^6.1.2	State management across 7 providers
supabase_flutter	^2.8.4	Auth, database, real-time, and storage client
google_sign_in	^6.2.1	Native Google OAuth sign-in
geolocator	^13.0.2	GPS coordinates for location and distance
geocoding	^3.0.0	Reverse geocoding coordinates to city name
url_launcher	^6.3.1	Opening Google Maps for pharmacy directions
cached_network_image	^3.4.1	Efficient network image loading and caching
image_picker	^1.1.2	Camera and gallery for prescription photos
flutter_local_notifications	^18.0.1	In-app notification banners
intl	^0.19.0	Date, time, and currency formatting
shared_preferences	^2.3.3	Onboarding seen flag and local settings
cupertino_icons	^1.0.8	iOS-style icons

3.2.2. Backend and Cloud Services

The table below lists the cloud services used by PharmaVault and the role each service plays in the application.

Service	Purpose
Supabase (PostgreSQL)	Cloud-hosted relational database with 10 tables storing all application data
Supabase Auth	Email and password authentication and Google OAuth
Supabase Realtime	Live subscriptions on the orders and prescriptions tables for notification triggering
Supabase Storage	prescriptions bucket storing uploaded prescription images
Google Cloud Console	Web OAuth client ID for Google Sign-In via serverClientId

3.3. Implementation Description

Authentication Flow

The authentication system handles three distinct flows. Each is described below with the specific Supabase method called and the edge cases handled by the implementation.

Email and password registration: Supabase.auth.signUp() creates the auth user. A profile row is immediately upserted to the profiles table using upsert rather than insert to handle the auto-trigger that creates a partial row. Email confirmation is disabled for development. On success, the user is navigated directly to the main screen.

Email and password login: signInWithPassword() is called. On success, the cart is fetched and the Supabase Realtime subscription for notifications is started.

Google Sign-In: GoogleSignIn(serverClientId: webClientId).signIn() retrieves an ID token. This is passed to supabase.auth.signInWithIdToken(provider: OAuthProvider.google, idToken: ...). No google-services.json or Google Services Gradle plugin is required as only the web OAuth client ID is needed. On first sign-in, a profile row is created.

Product and Pharmacy Data Flow

Products are fetched from the products table with joined brands and categories data. Pharmacies are fetched from the profiles table filtered by customer_type equal to pharmacy. The pharmacy_products table links each pharmacy to its own product subset with pharmacy-specific pricing and stock counts. Product detail queries pharmacy_products joined with profiles to show which pharmacies carry that medicine and at what price, ordered cheapest first.

Cart and Order Flow

The cart and order flow follows the steps described below.

- The user taps Add to Cart and a cart row is inserted or the quantity is incremented in Supabase
- The Cart screen fetches all cart rows with product joins
- The Checkout screen collects the delivery address, city, and contact details
- OrderProvider.createOrder() fetches the cart, calculates the total, generates an invoice number formatted as INV-YEAR-MILLISECONDS, inserts the order row, inserts

order_items rows, clears the cart, and fires a local notification confirming order placement
- The order history is fetched from the orders table filtered by c_id equal to auth.uid()

GPS and Location Flow

Location is resolved at app startup through the following steps.

- LocationProvider.fetchLocation() is called at app startup via a cascade in main.dart
- The app checks whether the location service is enabled and requests permission if needed
- On grant, Geolocator.getCurrentPosition() retrieves coordinates and placemarkFromCoordinates() reverse geocodes them to a city name
- The home screen header displays the real city name and falls back to Accra, Ghana if permission is denied
- Pharmacy cards display distance using Geolocator.distanceBetween() comparing the user position to the pharmacy lat and lng stored in profiles
- The Directions button in pharmacy detail launches the Google Maps navigation URL via url_launcher

Notification Flow

In-app notifications are delivered through the following pipeline.

- NotificationService.initialize() runs at app startup, creates the Android notification channel with high importance, and requests POST_NOTIFICATIONS permission on Android 13 and above
- After login or registration, NotificationProvider.subscribe(userId) opens two Supabase Realtime channels: one for orders table UPDATE events and one for prescriptions table UPDATE events, both filtered to the authenticated user's ID
- Order placement triggers a local notification immediately
- The Notifications screen reads from NotificationProvider.items, showing a real-time list with unread indicators, timestamps, and a mark-all-read option

The figure below illustrates the LangGraph-style backend processing pipeline adapted for PharmaVault's notification and order flow.

[IMAGE PLACEHOLDER]
Fig 5: PharmaVault Notification and Order Processing Flow

Fig 5. Notification and Order Processing Flow

3.4. Key Design Decisions and Challenges

The table below documents the key design decisions made during development and the challenges encountered, together with the resolutions applied.

Decision / Challenge	Resolution
No separate backend server	PharmaVault runs entirely on Supabase. All queries and business logic are expressed as Supabase client calls with PostgreSQL-level RLS policies enforcing data isolation. This eliminated deployment complexity and infrastructure cost.
Pharmacy-product linking table	The pharmacy_products table was introduced after initial development showed that the pharmacy detail screen was incorrectly showing all products to all pharmacies. The linking table gives each pharmacy its own product subset, price, and stock count.
upsert for profile creation	Supabase's on_auth_user_created trigger creates a partial profile row immediately when a user signs up. The initial insert call was failing with a primary key conflict. Changing to upsert resolved this silently, with the application data overwriting the empty trigger row.
Google Sign-In without google-services.json	The google_sign_in package supports a serverClientId parameter that allows Google OAuth to function using only the web client ID. The Google Services Gradle plugin is not needed. This was discovered after a build failure caused by attempting to add the plugin.
RLS blocking profile insert on signup	When Supabase email confirmation is enabled, auth.uid() returns null in the session immediately after signUp(), which caused the RLS policy to block the profile insert. This was resolved by disabling email confirmation for development.
FK constraint on pharmacy seed data	The profiles table has a foreign key referencing auth.users. Inserting seed pharmacy profiles with generated UUIDs failed because those UUIDs had no corresponding auth users. Resolved by dropping the FK constraint before seeding, since pharmacy profiles are platform-managed data.
Supabase Realtime for notifications	Rather than polling for order status changes, Supabase Realtime channels provide instant push-style updates while the app is open. The NotificationProvider opens one channel per table filtered to the authenticated user's ID.

3.5. Security Considerations

The table below documents the security measures implemented across key areas of the application.

Area	Implementation
Credentials	The Supabase anon publishable key is safe to expose client-side by design. The service role key is never included in the Flutter application.
Row Level Security	All 10 tables have RLS enabled. Customers can only read and write their own cart, orders, and prescriptions. Products and pharmacy_products are publicly readable with SELECT only.
Pharmacy data isolation	Pharmacy profiles are readable by all users, which is required for the browse feature, but only the pharmacy's own row is updatable.
Google Sign-In	Uses serverClientId only. No Firebase service account credentials are included in the application.
Input validation	All form fields are trimmed and validated client-side before any Supabase call is made.
Image uploads	Prescription images are stored in a private Supabase Storage bucket. Public read is intentionally disabled so only the authenticated user can access their own files.

3.6. Screens Implemented

The table below lists all screens implemented in PharmaVault together with a description of each screen's purpose and content.

Screen	Description
SplashScreen	Animated logo that checks the Supabase session and routes to Home or Onboarding
OnboardingScreen	First-time user introduction slides
LoginScreen	Email and password login with a Google Sign-In button
RegisterScreen	Full registration form collecting name, email, password, contact, and city
MainScreen	5-tab NavigationBar shell hosting Home, Search, Cart, Orders, and Profile
HomeScreen	White header with GPS city and notification bell, greeting, search bar, prescription upload banner, category chips, horizontal product cards with Rx badge, and pharmacy cards with distance and delivery tags
SearchScreen	Large Search title, tabbed Medicines and Pharmacies views, category filter chips, result count, and pull-to-refresh

Screen	Description
ProductDetailScreen	Product image, description, dosage, side effects, Available At section showing real pharmacies sorted cheapest first, quantity selector, and Add to Cart button
PharmacyDetailScreen	Pharmacy header with rating, hours, and Open badge, pharmacy-specific product list with prices and stock, Directions button, and Call button
CartScreen	Cart items with quantity controls, subtotal calculation, and checkout button
CheckoutScreen	Delivery address, city, and contact form with order confirmation
OrdersScreen	Order history list with invoice numbers, status badges, and totals
OrderDetailScreen	Itemised order breakdown, delivery details, and status timeline
UploadPrescriptionScreen	Camera and gallery picker with prescription image preview and upload to Supabase Storage
PrescriptionsScreen	List of uploaded prescriptions with status indicators: pending, reviewing, approved, or rejected
NotificationsScreen	Real-time notification history from Supabase Realtime with unread dots, mark-all-read, and empty state
ProfileScreen	View and edit name, contact, and city with logout functionality
SettingsScreen	App settings
HelpSupportScreen	Help and FAQ
SavedAddressesScreen	Saved delivery addresses
SavedPharmaciesScreen	Bookmarked partner pharmacies

Activity 4: Testing, Deployment and Presentation

4.1. Overview

PharmaVault is deployed entirely on Supabase cloud infrastructure. There is no self-hosted server. The Flutter application is distributed as an Android APK for testing on physical devices.

4.2. Deployment

4.2.1. Database and Auth

The cloud infrastructure for PharmaVault is hosted on Supabase as described below.

- Hosted on Supabase with managed PostgreSQL, Auth, Storage, and Realtime
- Project URL: https://sbjmzumtejbzwnrbjrdr.supabase.co
- Row Level Security enabled on all 10 tables
- prescriptions storage bucket created for prescription image uploads
- Realtime enabled on the orders and prescriptions tables

4.2.2. Mobile App

The table below documents the build artefacts and configuration used for the PharmaVault Android application.

Item	Details
Debug APK	build/app/outputs/flutter-apk/app-debug.apk
Release APK	build/app/outputs/flutter-apk/app-release.apk
Build Command	flutter build apk --release
Min SDK	Android 6.0 (API 23)
Target SDK	Android 14 (API 34)

4.3. Test Cases

The table below documents the test cases executed to verify the functionality of PharmaVault, including the feature tested, the test performed, and the expected result.

No.	Feature	Test	Expected Result
T1	Registration	Register with valid name, email, password, contact, and city	Account created, profile saved to Supabase, user navigated to Home
T2	Login	Login with correct credentials	User navigated to Home with name shown in greeting
T3	Login	Login with wrong password	Error snackbar displayed

No.	Feature	Test	Expected Result
T4	Google Sign-In	Tap Continue with Google on a physical device	Google account picker opens, user signs in, profile created
T5	Browse Products	Open Home screen	Products load from Supabase with names and prices
T6	Category Filter	Tap a category chip in Search	Product list filters to that category
T7	Pharmacy List	Open Search and navigate to Pharmacies tab	4 partner pharmacies shown with distance labels
T8	Pharmacy Detail	Tap a pharmacy card	Only that pharmacy's products shown at its specific prices
T9	Product Detail	Tap a product	Available At section shows real pharmacies carrying it, sorted cheapest first
T10	GPS Location	Open Home screen with location permitted	Header shows real city name such as Accra, Ghana
T11	Distance	View pharmacy cards	Distance from user shown in kilometres
T12	Directions	Tap Directions on pharmacy detail	Google Maps opens with navigation to pharmacy coordinates
T13	Add to Cart	Tap Add on a product	Product added to cart and cart badge increments
T14	Place Order	Complete checkout with delivery address	Order created with unique invoice number and Order Placed notification fires
T15	Order History	Open Orders tab	All previous orders shown with status badges
T16	Upload Prescription	Upload a prescription image	Image uploaded to Supabase Storage and appears in Prescriptions screen
T17	Notification - Order	Update order status in Supabase	In-app notification banner fires with correct status message
T18	Notification - Prescription	Update prescription status in Supabase	In-app notification banner fires
T19	Notifications Screen	Open Notifications	Real notification history shown with unread dots
T20	Pull to Refresh	Pull down on Search screen	Pharmacies and products re-fetched from Supabase

No.	Feature	Test	Expected Result
T21	Logout	Tap Logout in Profile	Session cleared and user returned to Login screen
T22	Data Persistence	Reinstall app and log in	Cart, orders, and prescriptions all restored from Supabase

Activity 5: Pending and Planned Features

5.1. Overview

The following features are planned but not yet implemented. They represent the next phase of development for PharmaVault.

Feature	Description
Payment Integration	Paystack or MTN Mobile Money for Ghana to allow patients to pay for orders in-app
Background Push Notifications	FCM integration to deliver notifications when the application is fully closed
Profile Photo Upload	Allow users to set a profile picture stored in Supabase Storage
Pharmacy Dashboard	A separate interface for pharmacy staff to manage their inventory and update order statuses
Order Tracking Timeline	A visual status timeline on order detail showing Placed, Confirmed, Dispatched, and Delivered stages
App Store Submission	Change the application ID from com.example.pharmavault_app, generate a production signing keystore, and submit to Google Play

Technology Stack Summary

The table below summarises the full technology stack used to build and deploy PharmaVault.

Category	Technology
Mobile Framework	Flutter 3.x (Dart 3.x)
State Management	Provider with 7 providers
Backend / Database	Supabase (PostgreSQL, Auth, Realtime, and Storage)
Authentication	Supabase Auth with email and password and Google OAuth
Location	Geolocator and Geocoding
Notifications	flutter_local_notifications and Supabase Realtime
Image Loading	cached_network_image
Navigation	Named routes and NavigationBar (Material 3)
Platform	Android, minimum API 23, target API 34

GitHub: https://github.com/Rose-Alice18/PharmaVault-Mobile-App

YouTube usually deletes my videos, so I always send the general link to the channel just in case: www.youtube.com/@roselinetsatsu5075

PharmaVault - Your Digital Pharmacy Companion